

Case 2 — SRT Common Factor Risk Modeling

A common-factor risk framework, an illustrative five-year Monte Carlo model, and a concise proposal for tail-risk measurement and stress testing.

1. Original Question

Case 2: SRT Common Factor Risk Modeling

Context: A Synthetic Risk Transfer (SRT) is a transaction where banks transfer credit risk of a loan portfolio to investors without selling the loans. You're analyzing a first-loss tranche (0%-10% attachment) on a corporate credit portfolio with a 5-year scheduled maturity, quarterly replenishment, no early amortization triggers, paying S+700bps.

Task: Develop an approach for modeling common factor risks.

Questions to Address:

- What are the key common factor risks in this SRT?
- How would you measure/quantify these risks?
- What tail risks concern you most?
- How would you attempt to measure these tail risks?

Deliverables:

1. Risk framework outline (bullets or brief paragraphs)
2. Proposed measurement methodology
3. AI usage summary (if applicable): helpful prompts, rejected suggestions, validation approach

2. Risk factors and transmission channels

I would focus on three risk factors and the linked recovery channel:

Risk factor	Transmission channel	Modeling approach
Macro and credit	A downturn raises default likelihood across borrowers and worsens recoveries.	Use a shared macro state in both borrower credit scores and mean LGD.
Interest rates and refinancing	Higher rates increase debt-service and refinancing costs, raising PD.	Adjust reference PD for a rate shock, then simulate defaults and losses.
Sector concentration	Borrowers in the same sector share shocks; concentration leaves more of the pool exposed to them.	Use sector factors in credit scores; test initial sector PD assignments and concentration of replacement loans.
Recovery and liquidity	In a downturn, lower collateral values and difficulty selling distressed assets can reduce recoveries, raising LGD when defaults are already high.	Link mean LGD to the macro state. Distressed-asset liquidity is part of the economic rationale, but is not modeled separately.

Quarterly replenishment keeps exposure in the pool and can change its credit quality. With no early-amortization triggers, deterioration does not automatically stop that exposure. The contractual spread stays at +700bps even as risk increases.

I would simulate defaults and LGD together, allocate pool losses to the 0–10% first-loss tranche, and compare expected loss, tail loss and tranche exhaustion across scenarios.

3. Risk Framework Outline

The diagram summarizes how the risk factors in Section 2 feed into defaults, recoveries and first-loss tranche risk.

From risk factors to tranche losses

Three channels in the initial model; macro conditions link defaults and loss severity.

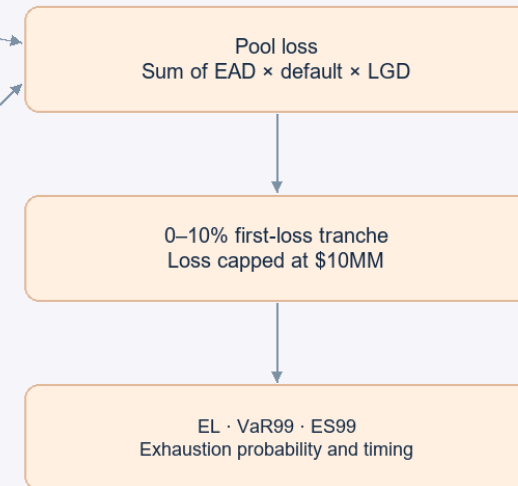
RISK FACTORS



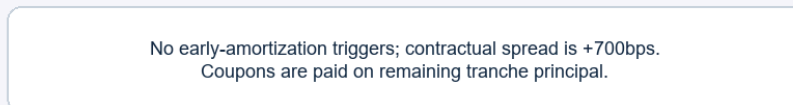
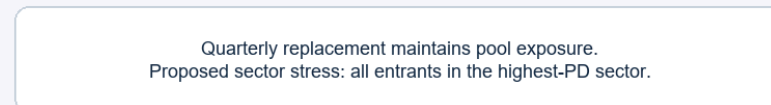
DEFAULTS AND RECOVERIES



LOSS MEASURES



REPLENISHMENT AND CONTRACT TERMS



Baseline replacements keep the same sector. Sector concentration tests are proposed, not run.

The shared macro state links defaults and LGD. Sector factors affect defaults; a direct sector effect on LGD is left out of this initial model.

4. Proposed Measurement Methodology

I would use a quarterly Monte Carlo model over the five-year transaction life. Starting with borrower PDs, sector assignments and exposures, I would simulate shared macro and sector states, generate defaults, and draw LGD conditional on the macro state. Each quarter, I would allocate losses to the first-loss tranche and replace exiting loans under the assumed replenishment rule. The resulting paths give expected losses, tail losses, and the probability and timing of tranche exhaustion.

I would then compare these measures with the baseline under the three stress tests in Section 7. The example below uses assumed inputs to demonstrate the method; applying it to the actual portfolio would require calibrating PDs, recoveries, factor loadings and persistence to data.

4.1 Start with expected loss

For one loan over a specified period, with fixed exposure:

$$EL = PD \times LGD \times EAD. \quad (4.1)$$

Here **PD** is the probability of default, **LGD** is the mean loss fraction conditional on default, and **EAD** is exposure at default in dollars. Their product gives expected loss (**EL**) in dollars.

The relevant LGD is the mean **among defaults** (about 58% in this simulation), not the 40% neutral-state mean. Expected loss alone does not capture how defaults and severe losses cluster.

4.2 How the channels affect losses

Arrows show the direction under adverse conditions; $\leftrightarrow$ means exposure is maintained under the assumed replenishment rule.

Channel	Main effect	Why it matters
Macro and credit	PD $\uparrow$ · LGD $\uparrow$	More defaults coincide with worse recoveries.
Interest rates and refinancing	PD $\uparrow$	Higher debt-service and refinancing costs raise default likelihood.
Sector concentration	Default clustering $\uparrow$; pool PD $\uparrow$ if shifted toward higher-PD sectors	More exposure shares the same sector shock.
Recovery and liquidity	Recovery $\downarrow$ · LGD $\uparrow$	Each default produces a larger loss; the model captures this through macro-linked LGD.
Quarterly replenishment; no early-amortization triggers	Pool EAD $\leftrightarrow$	Exposure continues through stress; weaker replacements can also raise pool PD.

The recovery row describes the loss-severity side of the macro channel. It is not an additional independent factor in the simulation.

4.3 Simulate and allocate losses

Simulate macro and sector states each quarter, use them to generate borrower defaults, and draw LGD for each default. Sum the loan losses and allocate them to the first-loss tranche:

$$L_t = \sum_{q=1}^t \sum_i EAD_{i,q} D_{i,q} LGD_{i,q},$$
$$TL_t = \min\{\max(L_t - A, 0), B - A\}.$$
(4.2)

$D_{i,q}$ is 1 if the borrower in loan slot i defaults in quarter q , and 0 otherwise; $LGD_{i,q}$ is its realized loss fraction. L_t and TL_t are cumulative pool and tranche losses through quarter t . The attachment and detachment amounts are $A = \$0$ and $B = \$10MM$ for the assumed \$100MM pool. All loss amounts are in dollars.

4.4 Measure the outcomes

Use five-year losses to calculate EL, VaR99 and ES99, plus the probability and timing of tranche exhaustion. Report pool tail losses as well as tranche losses: the tranche's 100% loss cap can hide further deterioration in the pool. Section 5 sets out the model, Section 6 reports the baseline results, and Section 7 proposes three stress tests.

5. Model assumptions and mechanics

5.1 Tail risk that concerns me most

The main concern is joint deterioration in PD and LGD: a shared downturn causes more defaults and worse recoveries at the same time, amplifying losses and potentially exhausting the first-loss tranche.

5.2 Model assumptions

Transaction assumptions

- **First-loss tranche:** attachment 0%, detachment 10%.
- **Scheduled maturity:** five years.
- **Replenishment:** quarterly.
- **Early amortization:** no early-amortization triggers.
- **Coupon:** base rate S + 700bps (a 7-percentage-point spread).

Simulation assumptions

The following inputs and mechanics are assumed for the simulation; they are not supplied or calibrated by the case.

1. **Portfolio size:** a \$100MM pool and \$10MM tranche, with 100 equally sized borrower slots of \$1MM each.
2. **Sector mix and initial PD:** five sectors weighted 30%, 25%, 20%, 15% and 10%, with annualized reference PD parameters of 1.2%, 1.6%, 2.0%, 2.5% and 3.0%, respectively.
3. **Common factors and sensitivities:** let:
 - $Z_t \sim \mathcal{N}(0, 1)$ be the **macro state**, shared by all borrowers.
 - $U_{s,t} \sim \mathcal{N}(0, 1)$ be the **sector state**, shared by borrowers in sector s .
 - $\epsilon_{i,t} \sim \mathcal{N}(0, 1)$ be the **borrower-specific shock** for borrower i .

Here t indexes quarters, i indexes loan slots, and $s(i)$ identifies the sector of the borrower currently in slot i . The components are independent within each quarter; macro and sector persistence is specified in assumption 4. Borrower i 's credit-health score is:

$$X_{i,t} = -0.40Z_t - 0.30U_{s(i),t} + \sqrt{0.75}\epsilon_{i,t}. \quad (5.1)$$

Here $X_{i,t}$ is a model credit-health score, not an observed rating or PD. Higher common states mean worse conditions; a lower X makes default more likely.

Using the unit variances and independence specified above, $\text{Var}(X) = 0.40^2 + 0.30^2 + 0.75 = 1$. The coefficients imply **16% macro, 9% sector and 75% borrower-specific shares of score variance**.

Link to assumption 2: convert annualized PD to quarterly PD, $p_{i,q} = 1 - (1 - p_{i,a})^{1/4}$. A borrower defaults when $X_{i,t} < \Phi^{-1}(p_{i,q})$: PD sets the cutoff, while the factors generate the score.

4. **Factor persistence:** both the macro state and each sector state follow quarterly AR(1) processes with persistence $\phi = 0.65$:

$$\begin{aligned} Z_{t+1} &= 0.65Z_t + \sqrt{1 - 0.65^2}\eta_{t+1}, \\ U_{s,t+1} &= 0.65U_{s,t} + \sqrt{1 - 0.65^2}\xi_{s,t+1}. \end{aligned} \quad (5.2)$$

where η and ξ are fresh, independent standard-normal shocks. Initial states are also standard normal. **Gaussian** means normal random shocks; **stationary** means the baseline states retain mean 0 and variance 1 over time, not that their realized values stay constant. The **0.65** coefficient carries part of the current state into next quarter; the $\sqrt{1 - 0.65^2}$ coefficient keeps its variance at 1.

Borrower-specific shocks do not follow AR(1): $\epsilon_{i,t}$ is drawn independently each quarter and across borrowers. The total credit-health score can still persist because its macro and sector components persist.

5. **LGD assumption:** conditional on default and the macro state, draw the fraction lost from a Beta distribution:

$$LGD_{i,t} \mid \{\text{default}, Z_t\} \sim \text{Beta}(\kappa m_t, \kappa(1 - m_t)). \quad (5.3)$$

- **Distribution choice:** Beta keeps LGD between 0 and 1 and allows different shapes.
- **Mean m_t :** average LGD for defaults in a given macro state; the neutral-state mean is 40%.
- **Precision $\kappa = 5$:** controls dispersion; higher values give a tighter distribution. At a 40% mean, Beta(2,3) has a standard deviation of 20 percentage points. The choice of 5 is illustrative, not calibrated.
- **Downturn sensitivity $\gamma = 0.65$:** sets the response of mean LGD to the macro state:

$$m_t = \text{logistic}\{\text{logit}(0.40) + \gamma Z_t\}.$$

Moving Z_t from 0 to 1 raises mean LGD from 40% to approximately 56.1%. The same Z_t drives defaults in Equation (5.1), linking higher default likelihood with higher LGD. γ controls this link; it is separate from persistence ϕ .

- Interest-rate-driven PD adjustment:** assume each 100bps rate increase raises annual PD log-odds by 0.130. For example, a 2.0% reference PD rises to approximately 2.27% (about 2.3%). This sensitivity is illustrative, not calibrated.
- Loan turnover:** each surviving loan has an independent 5% probability of scheduled maturity each quarter.
- Replacement rule:** replace defaulted and matured face amounts at quarter-end with new borrowers in the same sector and at that sector's reference PD, effective next quarter, maintaining the \$100MM pool. This is an assumed replenishment mechanic, not an additional given term.
- Loss settlement:** recognize ultimate LGD immediately; cumulative losses never reset, and lost tranche principal is never replenished.
- Investor cash flows:** assume a flat base rate $S = 3\%$, giving a 10% annual coupon before stress. Pay quarterly on post-loss remaining tranche principal and return surviving principal at year five.
- Discounting:** use the assumed 3% benchmark to calculate expected discounted cash flows; this is not a market fair-value calibration.

5.3 Model specification

Default generation

For the proposed macro stress, replace Z_t in Equation (5.1) by $Z_t + h_t$:

$$X_{i,t} = -b_G(Z_t + h_t) - b_S U_{s(i),t} + \sqrt{1 - b_G^2 - b_S^2} \epsilon_{i,t}, \quad b_G = 0.40, \quad b_S = 0.30.$$

The baseline has $h_t = 0$. The code phases a macro shift in over the first four quarters and then holds it constant. Higher macro or sector states lower the credit score and make default more likely.

An active borrower defaults when $X_{i,t} < \Phi^{-1}(p_{i,q})$, where Φ^{-1} is the standard-normal inverse CDF. Here the subscripts a and q on PD denote annual and quarterly frequency. The quarterly threshold parameter is $p_{i,q} = 1 - (1 - p_{i,a})^{1/4}$, using the annualized reference PD $p_{i,a}$. These reference PDs set the cutoff; default probabilities conditional on the shared states vary with those states.

Interest rates and PD

$$\text{logit}(p_{i,a}^{\text{stress}}) = \text{logit}(p_{i,a}) + 0.130 \Delta r_{pp}. \quad (5.4)$$

The assumed coefficient **0.130** combines floating-debt and refinancing effects; it is not calibrated. Under a rate shock, use $p_{i,q} = 1 - (1 - p_{i,a}^{\text{stress}})^{1/4}$ in the default cutoff above.

Here $\Delta r_{pp} = 1$ means +100bps and $\text{logit}(p) = \ln[p/(1-p)]$. Each 100bps increase multiplies default odds by $e^{0.130} \approx 1.139$ (about +14%); a 2.0% PD becomes 2.27%, an increase of 0.27 percentage points.

LGD generation

$$m_t = \text{logistic}\{\text{logit}(0.40) + \gamma(Z_t + h_t)\},$$

$$LGD_{i,t} \mid \{\text{default, state}\} \sim \text{Beta}(\kappa m_t, \kappa(1 - m_t)), \quad \gamma = 0.65, \quad \kappa = 5. \quad (5.5)$$

Here $\text{logistic}(x) = 1/(1 + e^{-x})$. First, calculate mean LGD m_t from the macro state. At $Z_t + h_t = 0$, it is 40%; with $\gamma = 0.65$, a one-standard-deviation deterioration raises it to about 56%. Then, for each defaulted loan, draw its actual LGD from the Beta distribution around m_t , with precision $\kappa = 5$.

The same $Z_t + h_t$ lowers the credit score in default generation and raises mean LGD: worse conditions therefore bring more defaults and larger losses per default. LGD has no direct sector-factor term in this model. PD and realized LGD are not perfectly correlated: defaults also depend on sector and borrower shocks, and LGD has its own Beta randomness. Defaults are concentrated in adverse states, so average LGD among simulated defaults is about 58%, above the 40% neutral-state mean. All inputs are illustrative.

Loss allocation

Use dollar amounts here: the initial pool is \$100MM, with attachment $A = \$0$ and detachment $B = \$10MM$.

$$L_t = \sum_{q=1}^t \sum_i EAD_{i,q} \times D_{i,q} \times LGD_{i,q},$$

$$TL_t = \min\{\max(L_t - A, 0), B - A\} = \min\{L_t, \$10MM\},$$

$$N_t = (B - A) - TL_t = \$10MM - TL_t. \quad (5.6)$$

where:

- $EAD_{i,q}$ is the dollar exposure in loan slot i in quarter q ; $LGD_{i,q}$ is the realized loss fraction.
- $D_{i,q}$ equals 1 if loan i defaults in quarter q , and 0 otherwise.
- L_t is cumulative **pool loss**, TL_t cumulative **tranche loss**, and N_t the **remaining tranche principal**, all in dollars.

The first-loss tranche absorbs pool losses up to \$10MM. For example, a \$6MM pool loss leaves \$4MM of tranche principal; a \$12MM pool loss exhausts the tranche, whose loss is capped at \$10MM.

Pool loss percentage is $L_t/\$100MM$; tranche loss percentage is $TL_t/\$10MM$. The code also uses dollar amounts internally.

In [1]:

```
from pathlib import Path
import sys, platform, json
import numpy as np
import pandas as pd
# ndtr / ndtri: normal CDF / inverse CDF; expit / logit: probability / log-odds conversion.
from scipy.special import ndtr, ndtri, expit, logit
import matplotlib.pyplot as plt
from IPython.display import display, Markdown

# Plot appearance and folder for saved tables/charts.
plt.rcParams.update({'figure.dpi': 120, 'axes.spines.top': False, 'axes.spines.right': False})
OUTPUT = Path('output/case2'); OUTPUT.mkdir(parents=True, exist_ok=True)
SEED = 20261004 # Random seed: makes the simulation reproducible.
LGD_PRECISION = 5.0 # Beta precision kappa: controls dispersion, not mean LGD.
N_BASE = 20000 # Number of simulated five-year portfolio paths.

# POOL_NOTIONAL: initial pool face in dollars ($100MM).
# ATTACHMENT / DETACHMENT: dollar loss boundaries ($0 / $10MM).
# TRANCHE_NOTIONAL: initial tranche principal ($10MM).
# Q: number of quarterly simulation periods = 5 years x 4 quarters.
POOL_NOTIONAL = 100_000_000
ATTACHMENT, DETACHMENT = 0., 10_000_000.
TRANCHE_NOTIONAL = DETACHMENT - ATTACHMENT
Q = 20

# Assign each loan a sector ID (0-4): 30, 25, 20, 15 and 10 equal-sized loans.
# Because all loans are equal-sized, these counts also give sector exposure weights (%).
SECTOR = np.repeat(np.arange(5), [30,25,20,15,10])
N = len(SECTOR) # Number of loan slots: 100.
# Map each loan's sector ID to its initial annualized reference PD (e.g. 0.016 = 1.6%).
PD0 = np.array([.012,.016,.020,.025,.030])[SECTOR]
# EAD: dollar exposure per loan ($100MM / 100 = $1MM).
```

```
EAD = POOL_NOTIONAL / N
```

```
# Show software versions, simulation sizes and the sector/PD mapping.
print(f'Python {platform.python_version()}; NumPy {np.__version__}; pandas {pd.__version__}')
print(f'{N_BASE:,} five-year paths; seed {SEED}')
display(pd.DataFrame({'sector':np.arange(1,6), 'names':[30,25,20,15,10],
                      'annualized_PD_pct':[1.2,1.6,2,2.5,3]}))
```

Python 3.12.14; NumPy 2.5.3; pandas 3.0.6
20,000 five-year paths; seed 20261004

	sector	names	annualized_PD_pct
0	1	30	1.2
1	2	25	1.6
2	3	20	2.0
3	4	15	2.5
4	5	10	3.0

In [2]:

```
def tranche_loss(pool_loss, attachment=ATTACHMENT, detachment=DETACHMENT):
    """Allocate pool losses to the tranche, capped at its initial principal.

    pool_loss: pool loss ($), scalar or array of any shape.
    attachment, detachment: dollar loss boundaries, scalar defaults $0 / $10MM.
    Returns tranche loss ($), with the same shape as pool_loss.
    """
    return np.clip(np.asarray(pool_loss)-attachment, 0., detachment-attachment)

def var_es(x, alpha=.99):
    """Calculate empirical VaR and atom-aware expected shortfall.

    x: loss observations, shape (n_paths,), in dollars or loss fractions.
    alpha: scalar confidence level, e.g. 0.99 = 99%.
    Returns (VaR, ES): two scalar floats in the same units as x.
    """
    x = np.sort(np.asarray(x, dtype=float))
```

```

assert x.ndim == 1 and len(x)>0 and 0<alpha<1
n = len(x); boundary = alpha*n; j = int(np.floor(boundary))
var = x[int(np.ceil(boundary))-1]
es = ((j+1-boundary)*x[j] + x[j+1:].sum()) / (n-boundary)
return float(var), float(es)

def advance_factors(macro_state, sector_states, persistence, rng):
    """Update macro and sector states by one quarter using AR(1).

    macro_state: current Z, shape (n_paths,).
    sector_states: current U for five sectors, shape (n_paths, 5).
    persistence: scalar AR(1) coefficient; baseline 0.65 for both states.
    rng: random generator for new normal shocks.
    Returns updated macro_state and sector_states, with unchanged shapes.
    """
    innovation_scale = np.sqrt(1 - persistence**2)
    macro_state = persistence * macro_state + innovation_scale * rng.normal(size=len(macro_state))
    sector_states = persistence * sector_states + innovation_scale * rng.normal(size=sector_states.shape)
    return macro_state, sector_states

def quarterly_pd(annual_pd, rate_change_pp):
    """Adjust annual PD for interest rates and convert it to quarterly PD.

    annual_pd: reference probabilities (0.02 = 2%), shape (n_paths, n_loans).
    rate_change_pp: scalar rate shock in percentage points; 1 = +100bps.
    Returns quarterly probabilities, same shape as annual_pd, not score cutoffs.
    """
    # +100bps means rate_change_pp=1; assumed log-odds increase is 0.130.
    stressed_annual_pd = expit(logit(annual_pd) + rate_change_pp * (.15*.70 + .10*.25))
    # Numerically stable equivalent of 1 - (1 - annual PD)**(1/4).
    return -np.expml(np.log1p(-stressed_annual_pd) / 4)

def draw_credit_scores(sectors, macro_state, sector_states, macro_shift,
                      sector_shifts, bg, bs, rng):
    """Combine shared states and borrower shocks into credit-health scores.

    sectors: loan sector IDs 0-4, shape (n_paths, n_loans).
    macro_state, sector_states: Z (n_paths,) and U (n_paths, 5).
    macro_shift, sector_shifts: imposed stresses, scalar and shape (5,).
    bg, bs: scalar macro/sector score loadings, baseline 0.40 / 0.30.
    rng: random generator for borrower-specific shocks.

```

```

Returns credit scores X, shape (n_paths, n_loans); lower means weaker health.
"""
loan_sector_state = np.take_along_axis(sector_states, sectors, axis=1)
return (
    -bg * (macro_state[:, None] + macro_shift)
    -bs * (loan_sector_state + sector_shifts[sectors])
    +np.sqrt(1 - bg*bg - bs*bs) * rng.normal(size=sectors.shape)
)

def draw_defaults(active, sectors, macro_state, sector_states, pd_quarter,
                 macro_shift, sector_shifts, bg, bs, default_rng):
    """Identify defaults in the baseline Gaussian model.

    active, sectors: active-loan flags and sector IDs, shape (n_paths, n_loans).
    macro_state, sector_states: Z (n_paths,) and U (n_paths, 5).
    pd_quarter: quarterly probabilities, shape (n_paths, n_loans).
    macro_shift, sector_shifts: imposed stresses, scalar and shape (5,).
    bg, bs: scalar macro/sector score loadings; default_rng generates loan shocks.
    Returns Boolean flags, shape (n_paths, n_loans): True = default this quarter.
    """
    score = draw_credit_scores(sectors, macro_state, sector_states, macro_shift,
                               sector_shifts, bg, bs, default_rng)
    cutoff = ndtri(pd_quarter) # Normal inverse CDF converts quarterly PD to a score cutoff.
    return active & (score < cutoff)

def draw_default_lgd(defaults, macro_state, macro_shift, gamma, recovery_shift, kappa, rng):
    """Draw recovery severity for loans that default this quarter.

    defaults: Boolean default flags, shape (n_paths, n_loans).
    macro_state: Z per path, shape (n_paths,); macro_shift: scalar stress shift.
    gamma: scalar macro-LGD sensitivity; recovery_shift: scalar LGD log-odds shock.
    kappa: scalar Beta precision; rng: random generator for loan-level LGDs.
    Returns path_ids, lgd and adverse_state, each shape (K,), one per default.
    K counts this quarter's defaults across all paths, not the number of paths.
    path_ids are integer path IDs; lgd is a fraction; adverse_state is Z + macro_shift.
    """
    path_ids, _ = np.nonzero(defaults)
    adverse_state = macro_state[path_ids] + macro_shift
    mean_lgd = expit(logit(.4) + gamma * adverse_state + recovery_shift)

```

```

# Shared macro state links PD and LGD; Beta draws add loan-level recovery variation.
lgd = rng.beta(kappa * mean_lgd, kappa * (1 - mean_lgd))
return path_ids, lgd, adverse_state

def aggregate_loan_losses(path_ids, lgd, n_paths):
    """Sum this quarter's defaulted-loan losses separately for each path.

    path_ids: integer path ID for each default event, shape (K,).
    lgd: corresponding loan loss fractions, shape (K,); EAD is the fixed $1MM exposure.
    n_paths: scalar total number of paths; K is this quarter's default count.
    Returns new pool losses ($), shape (n_paths,); no-default paths get zero.
    """
    # bincount groups loan losses by simulation path, including paths with no defaults.
    return np.bincount(path_ids, weights=lgd * EAD, minlength=n_paths)

def quarterly_cashflows(pool_loss, quarter, benchmark_rate):
    """Calculate quarterly coupon and discounted cash-flow components.

    pool_loss: cumulative pool loss ($), shape (n_paths,).
    quarter: integer index, 0-19 for quarters 1-20.
    benchmark_rate: annual base/discount rate, e.g. 0.03 = 3%.
    Returns coupon ($), spread_annuity ($-years) and benchmark_pv ($),
    each with shape (n_paths,).
    """
    remaining = TRANCHE_NOTIONAL - tranche_loss(pool_loss)
    discount_factor = (1 + benchmark_rate/4)**(-(quarter + 1))
    # Pay coupon on principal remaining after this quarter's credit losses.
    coupon = remaining * (benchmark_rate + .07) / 4
    spread_annuity = remaining / 4 * discount_factor
    benchmark_pv = remaining * benchmark_rate / 4 * discount_factor
    return coupon, spread_annuity, benchmark_pv

def replenish_pool(active, defaults, sectors, annual_pd, replenish, rng):
    """Replace exiting loans at quarter-end using their sector's reference PD.

    active, defaults: loan-status flags, shape (n_paths, n_loans).
    sectors, annual_pd: sector IDs and reference PDs, same shape as active.
    replenish: Boolean replacement switch; rng generates scheduled maturities.
    Updates arrays in place; returns active, sectors, annual_pd with unchanged shapes.
    Replacement loans become exposed next quarter; tranche losses are not restored.
    """

```

```

"""
mature = (rng.random(active.shape) < .05) & active & ~defaults
exits = defaults | mature
active[exits] = False
replace = exits & replenish
sector_pd = np.array([.012, .016, .020, .025, .030])[sectors]
annual_pd[replace] = sector_pd[replace]
active[replace] = True
return active, sectors, annual_pd

def simulate(n=N_BASE, seed=SEED, macro=0., rate_pp=0., lgd_shift=0.,
            sector_shock=0., bg=.40, bs=.30, phi=.65, gamma=.65,
            replenish=True, controls=False, kappa=LGD_PRECISION):
    """Run the Gaussian model; optional inputs allow the proposed common-factor stresses.

    n, seed: path count and random seed; macro/sector_shock: adverse state shifts.
    rate_pp: rate shock in percentage points; lgd_shift: recovery log-odds shock.
    bg, bs, phi, gamma, kappa: model coefficients defined in the assumptions.
    replenish: replace exiting loans; controls: add fixed/shuffled LGD comparisons.
    Returns a dictionary: dollar histories (n, Q), dollar cash flows (n,),
    first-quarter default counts (n,) and scalar default-weighted mean LGD.
    """

    assert bg*bg + bs*bs < 1 and abs(phi) < 1 and kappa > 0
    factor_rng, default_rng, lgd_rng, maturity_rng, shuffle_rng = [
        np.random.default_rng(s) for s in np.random.SeedSequence(seed).spawn(5)
    ]
    sectors = np.broadcast_to(SECTOR, (n, N)).copy()
    z = factor_rng.normal(size=n)
    u = factor_rng.normal(size=(n, 5))
    pd_a = np.broadcast_to(PD0, (n, N)).copy()
    active = np.ones((n, N), bool)
    histories = {key: np.zeros((n, Q)) for key in
                 ['loss', 'gross', 'exposure', 'exposure_end', 'fixed', 'shuffled']}
    coupon = np.zeros(n)
    annuity = np.zeros(n)
    benchmark_pv = np.zeros(n)
    default_count, lgd_sum = 0, 0.
    benchmark_rate = .03 + rate_pp/100

    for t in range(Q):
        # 1. Update shared states; scenario shifts ramp in over the first four quarters.
        if t:

```

```

    z, u = advance_factors(z, u, phi, factor_rng)
    ramp = min((t + 1)/4, 1.)
    h = macro * ramp
    sector_shifts = np.zeros(5)
    sector_shifts[0] = sector_shock * ramp

# 2. Determine quarterly default events for active loans.
pq = quarterly_pd(pd_a, rate_pp)
defaults = draw_defaults(active, sectors, z, u, pq, h, sector_shifts,
                        bg, bs, default_rng)

counts = defaults.sum(axis=1)
histories['exposure'][t, :] = active.sum(axis=1) * EAD
if t == 0:
    q1_counts = counts.copy()

# 3. Generate severity and accumulate dollar losses across quarters.
path_ids, lgd, _ = draw_default_lgd(defaults, z, h, gamma, lgd_shift, kappa, lgd_rng)
histories['loss'][t, :] = (histories['loss'][t-1, :] if t else 0) + aggregate_loan_losses(path_ids, lgd, n)
histories['gross'][t, :] = (histories['gross'][t-1, :] if t else 0) + counts * EAD
if controls:
    histories['fixed'][t, :] = (histories['fixed'][t-1, :] if t else 0) + counts * EAD * .4
    # Shuffling keeps pool EL unchanged but breaks bad-state/high-LGD alignment.
    histories['shuffled'][t, :] = (histories['shuffled'][t-1, :] if t else 0) + aggregate_loan_losses(path_ids, shuffle_rng.permutation(lgd), n)
    default_count += len(lgd)
    lgd_sum += lgd.sum()

# 4. Calculate investor income on loss-reduced principal.
quarter_coupon, quarter_annuity, quarter_benchmark = quarterly_cashflows(histories['loss'][t, :], t, benchmark_rate)
coupon += quarter_coupon
annuity += quarter_annuity
benchmark_pv += quarter_benchmark

# 5. Replace eligible exits; entrants participate from next quarter.
active, sectors, pd_a = replenish_pool(active, defaults, sectors, pd_a, replenish, maturity_rng)
histories['exposure_end'][t, :] = active.sum(axis=1) * EAD

remaining = TRANCHE_NOTIONAL - tranche_loss(histories['loss'][t, :])
principal_pv = remaining * (1 + benchmark_rate/4)**(-Q)
result = {key: histories[key] for key in ['loss', 'gross', 'exposure', 'exposure_end']}
result.update(coupon=coupon, annuity=annuity, benchmark_pv=benchmark_pv,
             principal_pv=principal_pv, pv=benchmark_pv+.07*annuity+principal_pv,
             net_loss=TRANCHE_NOTIONAL-(coupon+remaining),
             mean_lgd=lgd_sum/default_count, q1_counts=q1_counts)

```

```

if controls:
    result.update(fixed=histories['fixed'], shuffled=histories['shuffled'])
return result

def metrics(loss):
    """Report five-year credit loss and exhaustion metrics.

    loss: cumulative pool losses ($), shape (n_paths, Q).
    Returns scalar percentages and median exhaustion year conditional on exhaustion.
    """
    terminal = loss[:, -1]
    tranche = tranche_loss(terminal) / TRANCHE_NOTIONAL
    pool_var, pool_es = var_es(terminal, .99)
    tranche_var, tranche_es = var_es(tranche, .99)
    hit = loss >= TRANCHE_NOTIONAL
    exhausted = hit.any(axis=1)
    median_year = np.median((hit.argmax(axis=1)[exhausted]+1)/4) if exhausted.any() else np.nan
    return {'Pool EL %':100*terminal.mean()/POOL_NOTIONAL,
            'Tranche EL %':100*tranche.mean(),
            'Pool VaR99 %':100*pool_var/POOL_NOTIONAL,
            'Pool ES99 %':100*pool_es/POOL_NOTIONAL,
            'Tranche VaR99 %':100*tranche_var,
            'Tranche ES99 %':100*tranche_es,
            'Exhaustion probability %':100*exhausted.mean(),
            'Median exhaustion year (conditional)':median_year}

```

6. Core simulation and tail-risk measures

Run 20,000 five-year quarterly paths and report:

- **Expected loss (EL):** average pool/tranche principal loss.
- **VaR99:** the 99th-percentile five-year credit loss.
- **ES99:** average loss in the worst 1% of outcomes, allowing for ties at the tranche's 100% cap.
- **Exhaustion probability:** the probability of losing the entire \$10MM tranche principal.
- **Exhaustion timing:** the median first-exhaustion year, conditional on exhaustion occurring.

Pool-loss percentages use initial pool face; tranche-loss percentages use initial tranche face. Report both: tranche VaR/ES may reach 100% and conceal differences in severe pool losses.

The second table compares the baseline with LGDs shuffled across defaults within each quarter. This preserves total pool loss across paths while breaking the pairing of each default with its macro-conditioned LGD, helping isolate its contribution to tail losses.

The cash-flow calculation also reports the spread that makes expected discounted coupons and surviving principal equal the initial tranche investment. This break-even spread uses the assumed benchmark discount rate; it is not a market price.

In [3]:

```
base = simulate(controls=True)
summary = pd.Series(metrics(base['loss']), name='Baseline')
display(summary.to_frame().round(3))
summary.to_csv(OUTPUT / 'baseline_risk.csv')

# A small dependence diagnostic: same defaults and same total expected pool loss.
dependence = pd.DataFrame({
    'Joint PD-LGD': metrics(base['loss']),
    'Shuffled LGD': metrics(base['shuffled'])
}).T[['Pool EL %', 'Pool ES99 %', 'Exhaustion probability %']]
display(dependence.round(3))
dependence.to_csv(OUTPUT / 'dependence_comparison.csv')

# Essential implementation checks; these do not constitute loan-data calibration.
assert EAD == 1_000_000 and TRANCHE_NOTIONAL == 10_000_000
assert np.allclose(tranche_loss([0, 6_000_000, 12_000_000]), [0, 6_000_000, 10_000_000])
assert np.allclose(var_es([0, 1, 2, 3], .625), (2, 8/3))
assert var_es([0, 0, 1, 1], .625) == (1., 1.)
assert np.isfinite(base['loss']).all() and (np.diff(base['loss'], axis=1) >= -1e-7).all()
assert (base['loss'] <= base['gross'] + 1e-7).all()
assert np.allclose(base['exposure'], POOL_NOTIONAL)
assert np.isclose(base['loss'][:, -1].mean(), base['shuffled'][:, -1].mean())
pd_quarter = quarterly_pd(PD0, 0.)
q1_standard_error = base['q1_counts'].std(ddof=1) / np.sqrt(N_BASE)
assert abs(base['q1_counts'].mean() - pd_quarter.sum()) < 3*q1_standard_error
```

	Baseline
Pool EL %	5.446
Tranche EL %	47.355
Pool VaR99 %	23.437
Pool ES99 %	30.225
Tranche VaR99 %	100.000
Tranche ES99 %	100.000
Exhaustion probability %	13.395
Median exhaustion year (conditional)	3.500

	Pool EL %	Pool ES99 %	Exhaustion probability %
Joint PD-LGD	5.446	30.225	13.395
Shuffled LGD	5.446	24.211	12.135

In [4]:

```
break_even = (TRANCHE_NOTIONAL-base['benchmark_pv'].mean()-base['principal_pv'].mean())/base['annuity'].mean()
display(Markdown(
    f"The baseline has **{summary['Tranche EL %']:.1f}% expected tranche principal loss** "
    f"and **{summary['Exhaustion probability %']:.1f}% exhaustion probability**. "
    f"Among exhausted paths, the median first-exhaustion time is "
    f"**{summary['Median exhaustion year (conditional)']:.1f} years**."
    f"The expected-cash-flow break-even spread is **{break_even*10000:.0f}bp**,"
    "above 700bp. This demanding hypothetical baseline requires PD, recovery and "
    "replenishment calibration before drawing a conclusion about the actual transaction."
))
```

The baseline has **47.4% expected tranche principal loss** and **13.4% exhaustion probability**. Among exhausted paths, the median first-exhaustion time is **3.5 years**. The expected-cash-flow break-even spread is **1288bp**, above 700bp. This demanding hypothetical baseline requires PD, recovery and replenishment calibration before drawing a conclusion about the actual transaction.

7. Proposed stress testing

I would start with three tests, changing one driver at a time and comparing the results with the baseline.

1. **Macro and credit:** worsen the shared macro state by one standard deviation. This raises both default likelihood and mean LGD. I would check how much pool ES99, expected tranche loss and exhaustion probability change, and whether exhaustion happens earlier.
2. **Interest rates and refinancing:** raise interest rates by 100bps and first look at the change in PD through the debt-service and refinancing channel. I would then carry the updated PDs through the simulation to the same loss and exhaustion measures. For cash-flow results, the floating coupon and discount rate would also change.
3. **Sector concentration:** vary the initial PDs assigned to sectors, including assigning the highest PD to the largest sector. Separately, put all replacement loans into the sector with the highest reference PD, using that sector's PD and common-factor exposure. I would track how the sector mix changes and compare the resulting tail losses and tranche exhaustion with the baseline.

These are proposed tests; the results above are from the baseline simulation.

8. Interpretation and limits

My main tail-risk concern is that PD and LGD rise together in bad conditions: more borrowers default just when recoveries are worse. This can produce large pool losses and exhaust the first-loss tranche.

The main limitations are:

- **Normal-distribution assumptions:** macro and sector states and borrower shocks are Gaussian, which may understate extreme, clustered losses.
- **Uncalibrated parameters:** the AR(1) persistence of 0.65, factor loadings, reference PDs and rate/LGD sensitivities are assumed rather than fitted to data. The numerical results depend on these choices.
- **Simplified portfolio and replenishment:** loans have equal exposures, and replacements follow a fixed rule. Actual borrower differences and replacement eligibility could change the results.
- **Immediate loss settlement:** the model recognizes ultimate LGD at default and leaves out recovery delays, which could affect cash flows and value.

9. AI usage

- **Helpful prompts:** I asked how to combine macro and sector factors in an individual borrower's credit score, and how to model the link between PD and LGD. AI helped develop the common-factor model and the shared macro channel for defaults and recoveries.
- **Rejected suggestions:** I rejected extra drift terms and numerous hardcoded parameters for sensitivity analysis and stress testing. I wanted to keep the initial model simple, with fewer assumptions and three focused stress tests.

- **Validation approach:** I used AI to question my assumptions and check whether the risk transmission, equations and code were consistent with my intended framework. I also reviewed implementation checks for loss allocation, expected shortfall and simulated default rates. This checks internal consistency, not empirical validity; calibration would require actual loan and recovery data.

In [5]:

```
metadata = {'seed':SEED, 'paths':N_BASE, 'quarters':Q,  
            'pool_notional':POOL_NOTIONAL, 'tranche_notional':TRANCHE_NOTIONAL,  
            'ead_per_loan':EAD, 'amount_unit':'dollars', 'lgd_precision':LGD_PRECISION,  
            'model':'Gaussian common-factor model; illustrative, not calibrated',  
            'break_even_spread_bp':float(break_even*10000)}  
(OUTPUT / 'run_metadata.json').write_text(json.dumps(metadata, indent=2), encoding='utf-8')  
print('Core results saved.')
```

Core results saved.